BeaconKit 아키텍처 분석 - 07. 실행 엔진 통합

1. 실행 엔진 통합 개요

BeaconKit은 **Engine API**를 통해 외부 실행 클라이언트(Geth, Reth 등)와 통합됩니다. 이는 합의 레이어(BeaconKit)와 실행 레이어(EL 클라이언트)를 분리하여 모듈성을 제공하면서도 이더리움 호환성을 유지하는 핵심 아키텍처입니다.

1.1 지원 실행 클라이언트

- **Bera-Geth**: Berachain 프로토콜의 공식 Go 구현체
- **Bera-Reth**: 성능과 안정성에 중점을 둔 Rust 기반 클라이언트
- **기타 Engine API 호환 클라이언트**: 표준 Engine API를 지원하는 모든 클라이언트

1.2 Engine API 핵심 메서드

```
type ExecutionEngine interface {
// 새로운 페이로드를 실행 클라이언트에 전송
NotifyNewPayload(ctx context.Context, req ctypes.NewPayloadRequest, retryOnSynci
// 포크 선택 업데이트 및 페이로드 빌딩 요청
NotifyForkchoiceUpdate(ctx context.Context, req *ctypes.ForkchoiceUpdateRequest)
// 빌드된 페이로드 검색
GetPayload(ctx context.Context, req *ctypes.GetPayloadRequest) (ctypes.BuiltExec
}
```

2. EngineClient 구조

2.1 클라이언트 구성

```
type EngineClient struct {
*ethclient.Client
// cfg는 엔진 클라이언트의 제공된 구성
cfg *Config
// logger는 엔진 클라이언트의 로거
logger log.Logger
// eth1ChainID는 실행 클라이언트의 체인 ID
eth1ChainID *big.Int
// clientMetrics는 엔진 클라이언트의 메트릭
metrics *clientMetrics
// capabilities는 실행 클라이언트가 가진 기능들의 맵
capabilities  map[string]struct{}
// connected는 실행 클라이언트에 성공적으로 연결되었을 때 true로 설정
connectedMu sync.RWMutex
connected  bool
}
```

2.2 클라이언트 초기화

```
func  New(
```

```
cfg *Config,
logger log.Logger,
jwtSecret *jwt.Secret,
telemetrySink TelemetrySink,
eth1ChainID *big.Int,
) *EngineClient {
ethClient := ethclientrpc.NewClient(
cfg.RPCDialURL.String(),
jwtSecret,
cfg.RPCJWTRefreshInterval,
)

// 최소 RPC 타임아웃 강제 적용
if  cfg.RPCTimeout < MinRPCTimeout {
logger.Warn("Automatically raising RPCTimeout",
"configured", cfg.RPCTimeout,
"minimum", MinRPCTimeout,
)
}
cfg.RPCTimeout = max(MinRPCTimeout, cfg.RPCTimeout)

return &EngineClient{
cfg: cfg,
```

```
logger: logger,
Client: ethclient.New(ethClient),
capabilities: make(map[string]struct{}),
eth1ChainID: eth1ChainID,
metrics: newClientMetrics(telemetrySink, logger),
connected: false,
}
}
```

2.3 연결 초기화 및 검증

```
func (s *EngineClient) Start(ctx context.Context) error {
// 클라이언트 시작
go  s.Client.Start(ctx)

s.logger.Info("Initializing connection to the execution client...",
"dial_url", s.cfg.RPCDialURL.String())

// 연결이 성공하면 초기화 루프를 건너뛸 수 있음
if  err := s.verifyChainIDAndConnection(ctx);  err == nil {
return  nil
}
```

```go
// 실행 클라이언트와의 연결 초기화 시도
ticker := time.NewTicker(s.cfg.RPCStartupCheckInterval)
defer ticker.Stop()
for {
select {
case <-ctx.Done():
return ctx.Err()
case <-ticker.C:
s.logger.Info("Waiting for execution client to start... 🍺🕔",
"dial_url", s.cfg.RPCDialURL)
if err := s.verifyChainIDAndConnection(ctx); err != nil {
if errors.Is(err, ErrMismatchedEth1ChainID) {
s.logger.Error(err.Error())
}
continue
}
s.connectedMu.Lock()
s.connected = true
s.connectedMu.Unlock()
return nil
}
}
```

```
}
```

3. Engine API 메서드 구현

3.1 NotifyForkchoiceUpdate

포크 선택 업데이트를 실행 클라이언트에 알리고 선택적으로 페이로드 빌딩을 요청합니다.

```
func (ee *Engine) NotifyForkchoiceUpdate(
ctx context.Context,
req *ctypes.ForkchoiceUpdateRequest,
) (*engineprimitives.PayloadID, error) {
var (
engineAPIBackoff = ee.newBackoff()
hasPayloadAttributes = req.PayloadAttributes != nil
)

return  backoff.Retry(ctx, engineAPIBackoff,
func() (*engineprimitives.PayloadID, error) {
// 로그 및 포크 선택 업데이트 호출
ee.metrics.markNotifyForkchoiceUpdateCalled(hasPayloadAttributes)
payloadID, err := ee.ec.ForkchoiceUpdated(
ctx, req.State, req.PayloadAttributes, req.ForkVersion,
)
```

```
// NotifyForkchoiceUpdate는 두 가지 상황에서 호출됨:
// 1. 페이로드 빌딩 (PrepareProposal 중 또는 ProcessProposal에서 낙관적으로)
// 2. FinalizeBlock
switch {
case err == nil:
ee.metrics.markForkchoiceUpdateValid(req.State, hasPayloadAttributes, payloadID)
return payloadID, nil

case errors.IsAny(err, engineerrors.ErrSyncingPayloadStatus):
ee.logger.Info("NotifyForkchoiceUpdate: EL syncing. Retrying...")
ee.metrics.markForkchoiceUpdateSyncing(req.State, err)
return nil, err

case client.IsNonFatalError(err):
ee.logger.Info("NotifyForkchoiceUpdate: EL returns non fatal error. Retrying..."
ee.metrics.markForkchoiceUpdateNonFatalError(err)
return nil, err

case errors.IsAny(err, engineerrors.ErrInvalidPayloadStatus):
ee.logger.Error("NotifyForkchoiceUpdate: EL returned invalid payload.")
ee.metrics.markForkchoiceUpdateInvalid(req.State, err)
return nil, backoff.Permanent(err)
```

```
case  client.IsFatalError(err):
ee.logger.Info("NotifyForkchoiceUpdate: EL returns fatal error.", "err", err)
ee.metrics.markForkchoiceUpdateFatalError(err)
return  nil, backoff.Permanent(err)


default:
ee.logger.Info("NotifyForkchoiceUpdate: EL returns unknown error.", "err", err)
ee.metrics.markForkchoiceUpdateUndefinedError(err)
return  nil, backoff.Permanent(err)
}
})
}
```

3.2 NotifyNewPayload

새로운 페이로드를 실행 클라이언트에 알립니다.

```
func (ee *Engine) NotifyNewPayload(
ctx context.Context,
req ctypes.NewPayloadRequest,
retryOnSyncingStatus  bool,
) error {
var (
```

```
engineAPIBackoff = ee.newBackoff()
payloadHash = req.GetExecutionPayload().GetBlockHash()
payloadParentHash = req.GetExecutionPayload().GetParentHash()
)

_, err := backoff.Retry(ctx, engineAPIBackoff,
func() (*common.ExecutionHash, error) {
ee.metrics.markNewPayloadCalled(payloadHash, payloadParentHash)
lastValidHash, err := ee.ec.NewPayload(ctx, req)

// NotifyNewPayload는 세 가지 상황에서 호출됨:
// 1. ProcessProposal 상태 전환
// 2. FinalizeBlock 상태 전환
switch {
case  err == nil:
ee.metrics.markNewPayloadValid(payloadHash, payloadParentHash)
return  lastValidHash, nil

case  errors.IsAny(err, engineerrors.ErrSyncingPayloadStatus, engineerrors.ErrAc
ee.logger.Info("NotifyNewPayload: EL returns non valid status. Retrying...", "er
ee.metrics.markNewPayloadAcceptedSyncingPayloadStatus(err, payloadHash, payloadP
// ProcessProposal 중에는 블록을 검증할 수 있어야 함
if  retryOnSyncingStatus {
```

```
return  nil, err
}
// FinalizeBlock 중에는 블록을 검증할 필요 없음
ee.logger.Warn("NotifyNewPayload: pushed new payload to SYNCING node.",
"error", err,
"blockNum", req.GetExecutionPayload().GetNumber(),
"blockHash", payloadHash)
return &common.ExecutionHash{}, nil


case  client.IsNonFatalError(err):
ee.logger.Info("NotifyNewPayload: EL returns non fatal error. Retrying...", "err
if  lastValidHash == nil {
lastValidHash = &common.ExecutionHash{}
}
ee.metrics.markNewPayloadNonFatalError(payloadHash, *lastValidHash, err)
return  nil, err


case  errors.Is(err, engineerrors.ErrInvalidPayloadStatus):
ee.logger.Error("NotifyNewPayload: EL returned invalid payload.")
ee.metrics.markNewPayloadInvalidPayloadStatus(payloadHash)
return  nil, backoff.Permanent(err)


case  client.IsFatalError(err):
```

```
ee.logger.Error("NotifyNewPayload: EL returns fatal error.", "err", err)
if  lastValidHash == nil {
lastValidHash = &common.ExecutionHash{}
}
ee.metrics.markNewPayloadFatalError(payloadHash, *lastValidHash, err)
return  nil, backoff.Permanent(err)


default:
ee.logger.Error("NotifyNewPayload: EL returns unknown error.", "err", err)
ee.metrics.markNewPayloadUndefinedError(payloadHash, err)
return  nil, backoff.Permanent(err)
}
})


return  err
}
```

3.3 GetPayload

빌드된 페이로드를 검색합니다.

```
func (ee *Engine) GetPayload(
ctx context.Context,
```

```
req *ctypes.GetPayloadRequest,

) (ctypes.BuiltExecutionPayloadEnv, error) {

return  ee.ec.GetPayload(

ctx, req.PayloadID,

req.ForkVersion,

)

}
```

4. 페이로드 빌딩 워크플로우

4.1 블록 생성 시 페이로드 빌딩

```
1. PrepareProposal 시작

2. ForkchoiceUpdate(head, safe, finalized, payloadAttributes) → payloadID

3. GetPayload(payloadID) → executionPayload + blobsBundle

4. 비콘 블록 조립

5. CometBFT에 제안 반환
```

4.2 블록 검증 시 페이로드 검증

```
1. ProcessProposal 시작

2. NewPayload(executionPayload, retryOnSyncing=true) → 검증

3. 검증 성공 시 ACCEPT 반환
```

```
4. 검증 실패 시 REJECT 반환
```

4.3 블록 최종화 시 상태 업데이트

```
1. FinalizeBlock 시작
2. NewPayload(executionPayload, retryOnSyncing=false) → 실행
3. ForkchoiceUpdate(newHead, safe, finalized) → 헤드 업데이트
4. 상태 커밋
```

5. 오류 처리 및 재시도 메커니즘

5.1 오류 분류

Non-Fatal Errors (재시도 가능):

- 네트워크 연결 오류
- 일시적인 RPC 오류
- 실행 클라이언트 과부하

Fatal Errors (재시도 불가):

- 잘못된 요청 형식
- 인증 실패
- 지원되지 않는 메서드

Syncing Status (조건부 재시도):

- ProcessProposal: 재시도 필요 (블록 검증 필수)
- FinalizeBlock: 재시도 불필요 (이미 합의됨)

5.2 백오프 전략

```
func (ee *Engine) newBackoff() backoff.Backoff {
return  backoff.NewExponentialBackoff(
ee.cfg.RPCRetryDelay, // 초기 지연
ee.cfg.RPCRetryDelay*32, // 최대 지연
ee.cfg.RPCTimeout, // 전체 타임아웃
2.0, // 지수 배수
)
}
```

특징:

- 지수 백오프로 재시도 간격 증가
- 최대 재시도 시간 제한
- 영구 오류 시 즉시 중단

6. JWT 인증

6.1 JWT 기반 보안

```
ethClient := ethclientrpc.NewClient(
cfg.RPCDialURL.String(),
jwtSecret,
cfg.RPCJWTRefreshInterval,
```

```
)
```

특징:

- 공유 JWT 시크릿을 통한 인증
- 주기적인 토큰 갱신
- 무단 접근 방지

6.2 JWT 설정

```
# config.toml

[beacon-kit.engine]

jwt-secret-path = "path/to/jwt.hex"

rpc-dial-url = "http://localhost:8551"

rpc-jwt-refresh-interval = "30s"
```

7. 메트릭 및 모니터링

7.1 Engine API 메트릭

```
type engineMetrics struct {

// NewPayload 메트릭

newPayloadCalls prometheus.Counter

newPayloadValid prometheus.Counter

newPayloadInvalid prometheus.Counter
```

```
newPayloadErrors prometheus.Counter

// ForkchoiceUpdate 메트릭

forkchoiceUpdateCalls prometheus.Counter

forkchoiceUpdateValid prometheus.Counter

forkchoiceUpdateInvalid prometheus.Counter

forkchoiceUpdateErrors prometheus.Counter

// 응답 시간 메트릭

apiCallDuration prometheus.Histogram

}
```

7.2 모니터링 포인트

성공률 추적:

- NewPayload 성공/실패율
- ForkchoiceUpdate 성공/실패율
- GetPayload 성공/실패율

성능 추적:

- API 호출 응답 시간
- 재시도 횟수 및 빈도
- 백오프 지연 시간

상태 추적:

- 실행 클라이언트 연결 상태
- 동기화 상태
- 체인 ID 일치성

8. 구성 및 최적화

8.1 타임아웃 설정

```
const (
MinRPCTimeout = 1 * time.Second  // 최소 RPC 타임아웃
DefaultRPCTimeout = 30 * time.Second
)
```

8.2 연결 설정

```
type Config struct {
RPCDialURL *url.URL
RPCTimeout time.Duration
RPCStartupCheckInterval time.Duration
RPCJWTRefreshInterval time.Duration
RPCRetryDelay time.Duration
}
```

8.3 성능 최적화

연결 풀링:

- HTTP 연결 재사용
- Keep-alive 설정

- 동시 요청 제한

캐싱 전략:

- 페이로드 ID 캐싱
- 응답 결과 캐싱
- 중복 요청 방지

9. 실행 클라이언트별 특성

9.1 Bera-Geth

특징:

- Go 언어 기반
- 이더리움 Geth 포크
- 안정성과 호환성 중시

최적화:

- 메모리 사용량 최적화
- 빠른 동기화 지원
- 디버깅 도구 풍부

9.2 Bera-Reth

특징:

- Rust 언어 기반
- 성능과 안전성 중시
- 모듈형 아키텍처

최적화:

- 높은 처리량
- 낮은 메모리 사용량
- 병렬 처리 최적화

10. 설계 원칙

10.1 모듈성

- 합의 레이어와 실행 레이어 분리
- 표준 Engine API 사용
- 다양한 실행 클라이언트 지원

10.2 안정성

- 포괄적인 오류 처리
- 재시도 메커니즘
- 연결 상태 모니터링

10.3 성능

- 비동기 처리
- 백오프 전략
- 메트릭 기반 최적화

10.4 보안

- JWT 기반 인증
- 체인 ID 검증
- 안전한 통신 채널

이 실행 엔진 통합은 BeaconKit이 다양한 실행 클라이언트와 안전하고 효율적으로 상호작용하면서도 이더리움 호환성을 유지하는 핵심 메커니즘입니다.